



Week 2 — Image Preprocessing & Advanced Filtering

☑ Assignment Released	☐
☑ Checkpoint Done	☐
📅 End Date	May 5, 2026
≡ Learning Objectives	Apply preprocessing pipelines, threshold images, remove noise, and enhance image quality
🕒 Phase	🚫 Phase 1 – Foundation
📅 Start Date	April 27, 2026
⚙️ Status	Not Started
≡ Topics Overview	Image Thresholding, Noise Removal, Morphological Operations, Histogram Equalization, Gamma Correction
# Week Number	2



★ STAR UNION AI ACADEMY — Week 2

Led by **MOHAMED WALEED** · Head of AI @ Star Union



27 Apr – 3 May 2026 · Phase 1: Foundation



Week 2 Objectives

- Apply image thresholding (global, adaptive, Otsu)
- Remove noise using blur and denoising techniques

- Perform morphological operations (erosion, dilation, opening, closing)
 - Enhance image contrast using histogram equalization
 - Build a complete image preprocessing pipeline
-

Topic 1: Image Thresholding

Explanation

What is Thresholding?

Thresholding converts a grayscale image into a **binary image** — pixels become either 0 (black) or 255 (white) based on a threshold value. It is one of the most fundamental segmentation tools in CV.

Types of Thresholding:

1. Global (Simple) Thresholding:

A single threshold value T is applied to every pixel:

- If `pixel > T` → set to 255 (foreground)
- If `pixel ≤ T` → set to 0 (background)

Problem: Sensitive to uneven lighting.

2. Adaptive Thresholding:

Instead of one global T , it computes a *local* threshold for each small region of the image. This handles uneven illumination much better. Two methods:

- `ADAPTIVE_THRESH_MEAN_C` — threshold = mean of neighborhood
- `ADAPTIVE_THRESH_GAUSSIAN_C` — threshold = Gaussian-weighted mean

3. Otsu's Thresholding:

Automatically finds the optimal threshold by minimizing intra-class variance between foreground and background. Works best on bimodal histogram images (two distinct peaks). No manual T required.

OpenCV Threshold Types:

- `THRESH_BINARY` — standard binary
- `THRESH_BINARY_INV` — inverted binary
- `THRESH_TRUNC` — values above T become T

- `THRESH_TOZERO` — values below T become 0
-

References

- [Add link: OpenCV Thresholding Tutorial — https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html]
 - [Add link: Otsu's Method Explained — Towards Data Science]
 - [Add link: Image Segmentation Basics — YouTube]
-

Summary

- Thresholding converts grayscale → **binary image**
 - Global thresholding uses one T for the whole image
 - **Adaptive thresholding** handles uneven lighting by computing local thresholds
 - **Otsu's method** automatically finds the best T
 - Result: clean separation of foreground from background
-

Theory Questions

1. What is the purpose of thresholding in image processing?
 2. When would you prefer adaptive thresholding over global thresholding?
 3. Explain how Otsu's method determines the optimal threshold.
 4. What does `THRESH_BINARY_INV` do, and when would you use it?
 5. What kind of histogram does Otsu's method work best on?
-

Coding Task 1

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

# 1. Global threshold
_, thresh_global = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# 2. Adaptive Mean
thresh_adaptive_mean = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)

# 3. Adaptive Gaussian
thresh_adaptive_gauss = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)

# 4. Otsu's
_, thresh_otsu = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

titles = ['Original', 'Global T=127', 'Adaptive Mean', 'Adaptive Gaussian', "Otsu's"]
images = [gray, thresh_global, thresh_adaptive_mean, thresh_adaptive_gauss, thresh_otsu]

plt.figure(figsize=(15, 6))
for i, (t, im) in enumerate(zip(titles, images)):
    plt.subplot(1, 5, i+1)
    plt.imshow(im, cmap='gray')
    plt.title(t)
    plt.axis('off')
plt.show()

```

Topic 2: Morphological Operations

Explanation

Morphological operations process images based on **shape**. They operate on binary images using a **structuring element** (a small shape, like a 3×3 square or circle) that probes the image.

Erosion:

Shrinks foreground objects. A pixel is set to 1 only if ALL pixels under the structuring element are 1. Use: removes small noise, thin features.

Dilation:

Expands foreground objects. A pixel is set to 1 if ANY pixel under the structuring element is 1. Use: fills small holes, thickens features.

Opening = Erosion → Dilation:

Removes small white noise from background while preserving object shape. Ideal for **noise removal**.

Closing = Dilation → Erosion:

Fills small holes inside objects. Ideal for **hole filling**.

Morphological Gradient:

Difference between dilation and erosion. Produces the **outline/edge** of binary objects.

References

- [Add link: OpenCV Morphological Transformations — https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html]
 - [Add link: Morphological Operations Explained — PyImageSearch]
-

Summary

- Morphological ops use a **structuring element** to probe binary images
 - **Erosion** shrinks; **Dilation** expands
 - **Opening** removes noise; **Closing** fills holes
 - Morphological gradient → object boundary detection
 - Essential preprocessing step before contour detection or OCR
-

Theory Questions

1. What is a structuring element?
2. Explain the difference between erosion and dilation with a visual example.
3. When would you use Opening vs Closing?

4. How does morphological gradient relate to edge detection?
5. What happens if you apply dilation many times on a binary image?

Coding Task 2

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Structuring element
kernel = np.ones((5, 5), np.uint8)

eroded = cv2.erode(binary, kernel, iterations=1)
dilated = cv2.dilate(binary, kernel, iterations=1)
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)

titles = ['Binary', 'Eroded', 'Dilated', 'Opened', 'Closed', 'Gradient']
images = [binary, eroded, dilated, opened, closed, gradient]

plt.figure(figsize=(18, 5))
for i, (t, im) in enumerate(zip(titles, images)):
    plt.subplot(1, 6, i+1)
    plt.imshow(im, cmap='gray')
    plt.title(t)
    plt.axis('off')
plt.show()
```

Topic 3: Histogram Equalization & Gamma Correction

Explanation

Image Histogram:

A histogram of an image shows the **frequency distribution** of pixel intensity values. A dark image has histogram peaks near 0; a bright image near 255; a low-contrast image has a narrow peak in the middle.

Histogram Equalization:

Redistributes pixel intensities to **span the full range (0–255)**, effectively increasing global contrast. Mathematically, it applies a cumulative distribution function (CDF) as a mapping function.

CLAHE (Contrast Limited Adaptive Histogram Equalization):

Applies equalization locally to small tiles rather than globally. Avoids over-amplifying noise. Preferred in medical imaging and face recognition.

Gamma Correction:

A power-law transformation: `output = inputγ`

- $\gamma < 1$ → brightens the image (lift shadows)
- $\gamma > 1$ → darkens the image (crush highlights)

Used to correct camera sensor nonlinearity.

References

- [Add link: OpenCV Histogram Equalization — https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html]
- [Add link: CLAHE Explained — PyImageSearch]
- [Add link: Gamma Correction Tutorial — LearnOpenCV]

Summary

- A histogram shows **pixel intensity distribution**
- Histogram equalization spreads intensities across 0–255 for **better contrast**
- **CLAHE** applies this locally — better for real-world images

- Gamma correction adjusts brightness using a power law
 - These techniques are critical **preprocessing steps** for CV pipelines
-

? Theory Questions

1. What does a histogram of a low-contrast image look like?
 2. How does histogram equalization improve image quality?
 3. What is the difference between global histogram equalization and CLAHE?
 4. If $\gamma = 0.5$, does the image get brighter or darker? Why?
 5. When would you use CLAHE over standard histogram equalization?
-

Coding Task 3

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Global histogram equalization
eq_global = cv2.equalizeHist(gray)

# CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
eq_clahe = clahe.apply(gray)

# Gamma correction
def gamma_correction(img, gamma):
    table = np.array([((i / 255.0) ** gamma) * 255
                      for i in range(256)]).astype('uint8')
    return cv2.LUT(img, table)

bright = gamma_correction(gray, 0.4)
dark = gamma_correction(gray, 2.5)
```



```

titles = ['Original', 'Hist Eq.', 'CLAHE', 'Gamma=0.4', 'Gamma=2.5']
images = [gray, eq_global, eq_clahe, bright, dark]

plt.figure(figsize=(15, 5))
for i, (t, im) in enumerate(zip(titles, images)):
    plt.subplot(1, 5, i+1)
    plt.imshow(im, cmap='gray')
    plt.title(t)
    plt.axis('off')
plt.show()

```

Weekly Checkpoint — CP2: Preprocessing Pipeline



Pass Mark: 70%

Theory (50 pts)

1. (15 pts) Compare global thresholding, adaptive thresholding, and Otsu's method — when to use each.
2. (10 pts) What is erosion? Give a practical use case.
3. (10 pts) Explain CLAHE and why it's better than global histogram equalization.
4. (15 pts) Describe a full preprocessing pipeline for a noisy, low-contrast binary document image.

Coding (50 pts)

Task: Build an Automated Image Preprocessing Pipeline

Write a function `preprocess(image_path)` that:

1. Loads the image
2. Converts to grayscale
3. Applies CLAHE for contrast enhancement

4. Denoises with Gaussian blur
 5. Applies Otsu's thresholding
 6. Performs morphological opening
 7. Returns and displays the result alongside the original
-



Week 2 Assignment

Title: Document Scanner Preprocessing Pipeline

Deadline: 3 May 2026

Description:

Build a preprocessing pipeline for a scanned document (or any text image):

1. Apply grayscale conversion
2. CLAHE contrast enhancement
3. Gaussian denoising
4. Adaptive thresholding
5. Morphological cleaning (opening + closing)
6. Save the final clean image

Required Concepts: Thresholding, morphology, histogram equalization

Submission Format: Python script + before/after image comparison

Grading Rubric:

- Pipeline correctness: 40 pts
 - Visual quality of result: 30 pts
 - Code documentation: 30 pts
-



STAR UNION AI ACADEMY · Led by **MOHAMED WALEED**, Head of AI @
Star Union